Deep Learning,
Introduction and Advanced Concepts

3. Generalization and Practical Techniques

Louis Bagot

January 8, 2026

Outline

1. Generalization and Regularization in Deep Learning
   - Definitions
   - Dropout
   - L2 regularization
2. Practical tricks and techniques in Deep Learning
   - Data augmentation
   - Transfer
   - Self-Supervision

Outline

1. **Generalization and Regularization in Deep Learning**
   - Definitions
   - Dropout
   - L2 regularization
2. Practical tricks and techniques in Deep Learning
   - Data augmentation
   - Transfer
   - Self-Supervision

Defining Generalization

Our true objective is generally not good performance on the training data, but to use the model on *data it hasn't seen before*:

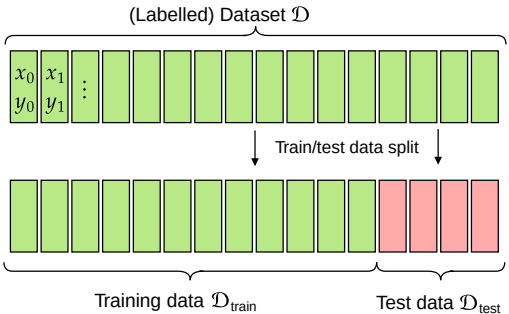
Generalization:

The ability to maintain good model performance beyond the training data.

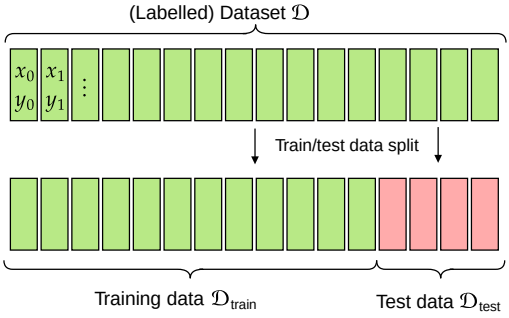
Evaluating Generalization: test data & loss

Is my model good at generalizing?

$\rightarrow$ let's **hide some data** from training and see how well it does on it!



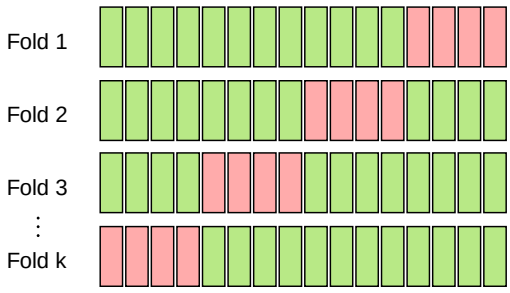
Evaluating Generalization: test data & loss



- Training: minimize loss/metrics on training data $\mathcal{D}_{\text{train}}$
- *Testing*: evaluate loss/metrics on test data $\mathcal{D}_{\text{test}}$ (every k steps)
  $\rightarrow$ the test loss is sometimes called *generalization error* or generalization loss.

k-fold cross-validation

Don’t have much data? Train & evaluate the model on many choices of test sets:



$\rightarrow$ Compute average performance over k trainings

Hyperparameter tuning

I want to find the set of hyperparameters that generalizes best.

First idea:

Hyperparameter tuning

I want to find the set of hyperparameters that generalizes best.

First idea:

> try out hyperparameters and pick the ones that did best on the test dataset!

Hyperparameter tuning

I want to find the set of hyperparameters that generalizes best.

First idea:

> try out hyperparameters and pick the ones that did best on the test dataset!

Problem:

Hyperparameter tuning

I want to find the set of hyperparameters that generalizes best.

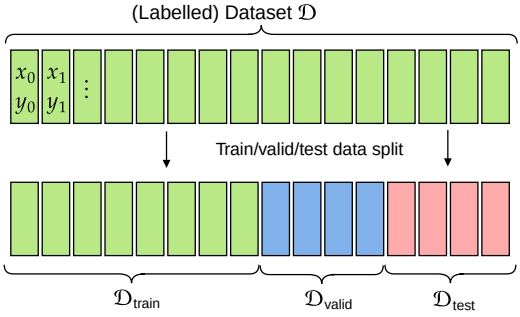
First idea:

try out hyperparameters and pick the ones that did best on the test dataset!

Problem:

it may be the hyperparameters that work best for *this specific* set of test data!

Validation set for hyperparameter search



Train on $\mathcal{D}_{\text{train}}$, optimize hyperparameters on $\mathcal{D}_{\text{valid}}$, evaluate on $\mathcal{D}_{\text{test}}$.

Regularization: reducing generalization error

Now we know what we need to do!

Regularization:

Any strategy to reduce test error, possibly at the expense of training error.

Generalization : what are the challenges?

Generalization:

The ability to maintain good model performance beyond the training data.

Generalization : what are the challenges?

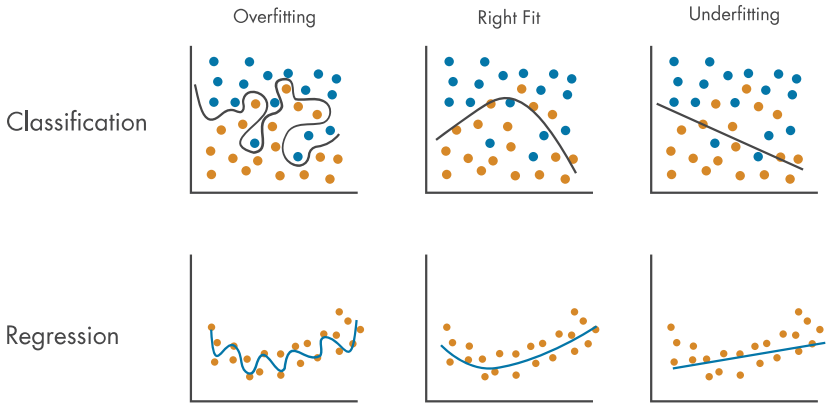
Generalization:

The ability to maintain good model performance beyond the training data.

Main issues:

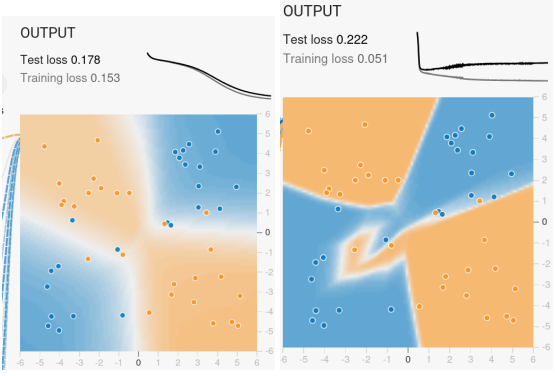
- *Overfitting*: the model learns the training data too closely, or by heart
- *Misaligned Data*: the training distribution is not representative of what we care about

Overfitting and Underfitting intuition



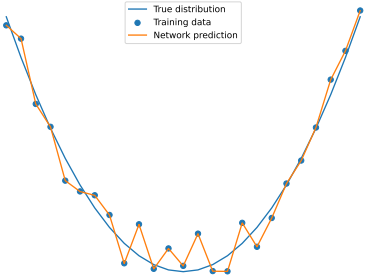
Overfitting concrete examples

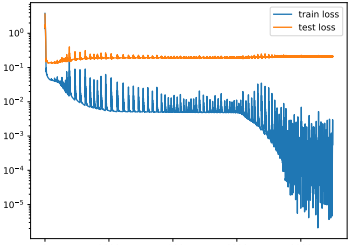
The model starts learning even the noise in the data! (tensorflow playground)



Left: an early good prediction. Right: with more training the network learns noise

Minimal overfitting example with a neural network

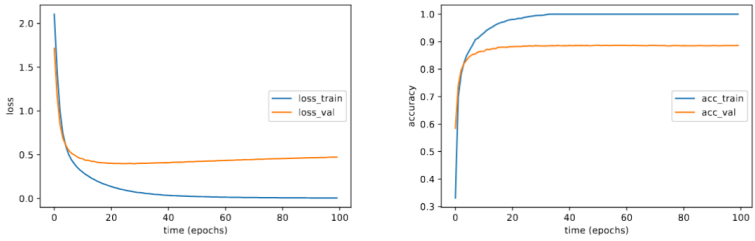




Overfitting viewed from the loss and accuracy

In the vast majority of cases you cannot visualize your whole function output

$\rightarrow$ **Visualize** train & test loss, accuracy! (and everything you can)

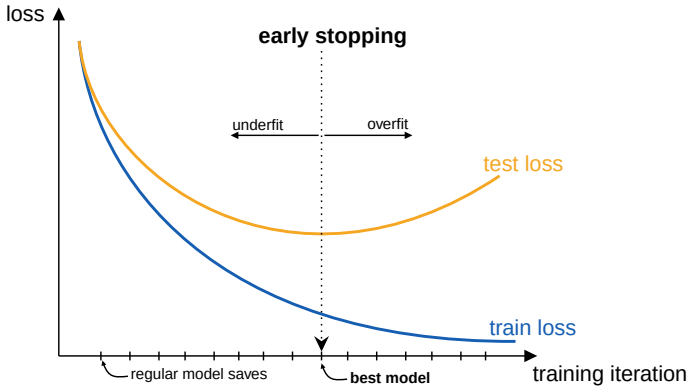


(a) (b)

Fig. 2: Overfitting of the model on MNIST dataset. (a) shows that after a certain epoch, the training loss gets decreased while validation loss gets increased. (b) shows the gap between training and validation accuracy over the time.

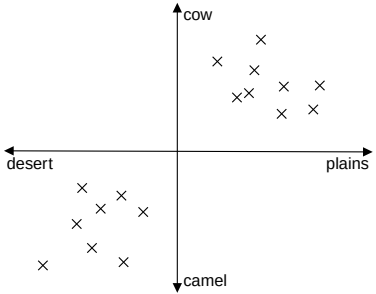
Early Stopping

Stop training before the model overfits!



Addressing Overfitting: Lazy solutions

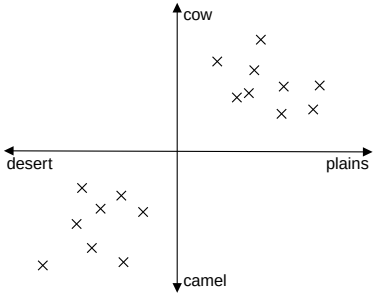
Imagine the network must distinguish cows from camels in pictures, with data distribution:

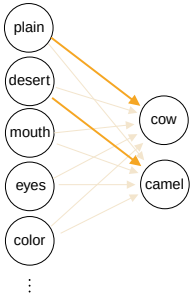


Problem:

Addressing Overfitting: Lazy solutions

Imagine the network must distinguish cows from camels in pictures, with data distribution:

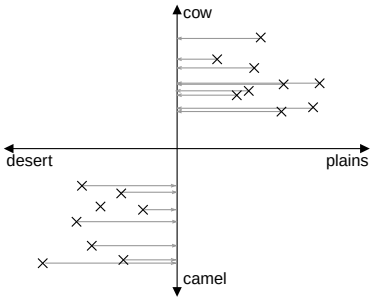


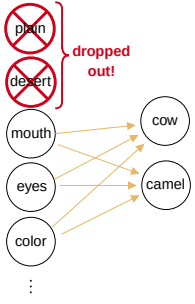


Problem: it's enough to identify the background to know the animal!

(much easier to do: average pixel value is either green or yellow)

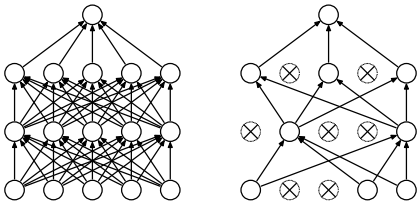
Solution: Regularization with Dropout





Randomly turn off features (e.g. 50%) $\rightarrow$ the network must learn to do without!

Dropout against Overfitting

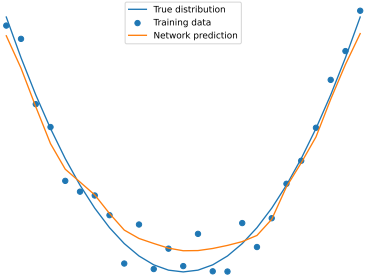


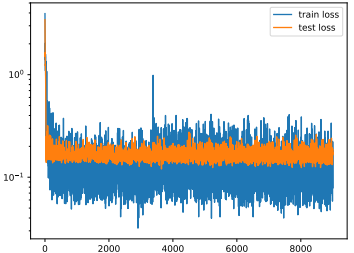
(a) Standard Neural Net (b) After applying dropout.

Figure 1: Dropout Neural Net Model. **Left**: A standard neural net with 2 hidden layers. **Right**: An example of a thinned net produced by applying dropout to the network on the left. Crossed units have been dropped.

$\rightarrow$ we can think of it as learning an *ensemble* of narrower networks.

Dropout $(p = 0.1)$ on the minimal example





Regularization Techniques

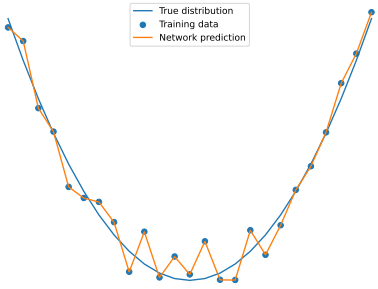
- Dropout
  - reduces overfit
  - easy to implement & no computational overhead
  - parameter p to fine-tune

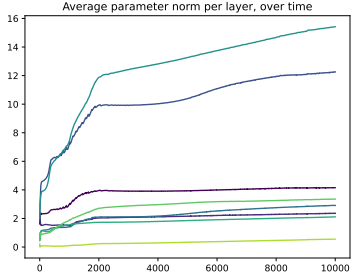
Regularization Techniques

- Dropout
  - reduces overfit
  - easy to implement & no computational overhead
  - parameter p to fine-tune
- **L2 regularization: weight decay**

Intuition for weight decay

Limiting model capabilities: fitting noise tends to require *high weights* $\boldsymbol{\theta}$



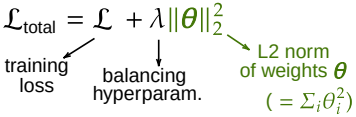


$\rightarrow$ intuition: **constrain weight size**!

i.e. force the net to use small θ_i

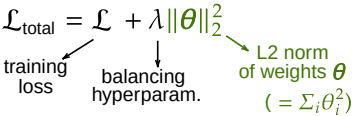
L2 Regularization: constraining weight size

Explicitly penalize big weights in the objective (loss) of the network:



L2 Regularization: constraining weight size

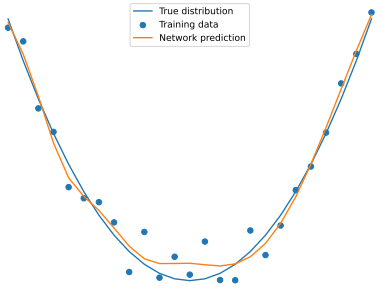
Explicitly penalize big weights in the objective (loss) of the network:

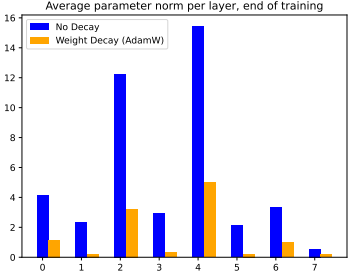


λ controls the balance:

- at $\lambda = 0$, the network simply solves its task (no changes)
- at $\lambda \to \infty$, the network only focuses on making each θ_i as small as possible
- in-between: balance the true objective with keeping weights small
  - Implementation in PyTorch: `AdamW(…, weight_decay=lambda)`

L2 Regularization on the minimal example





Regularization Techniques

- Dropout
  - reduces overfit
  - easy to implement & no computational overhead
  - parameter p to fine-tune
- L2 regularization: weight decay
  - reduces overfit
  - easy to implement & no computational overhead
  - parameter λ to fine-tune

$\rightarrow$ can generally use both + other methods

Outline

1. Generalization and Regularization in Deep Learning
   - Definitions
   - Dropout
   - L2 regularization
2. **Practical tricks and techniques**
   - Data augmentation
   - Transfer
   - Self-Supervision

The data problem

In 99% of cases, the main issue with using Deep Learning will be **data**

The data problem

In 99% of cases, the main issue with using Deep Learning will be **data**

- I don’t have data
  - good luck!

The data problem

In 99% of cases, the main issue with using Deep Learning will be **data**

- I don’t have data
- I have a small dataset

The data problem

In 99% of cases, the main issue with using Deep Learning will be **data**

- I don't have data
- I have a small dataset
  - my model is probably going to overfit
  - my model can't find relevant patterns

The data problem

In 99% of cases, the main issue with using Deep Learning will be **data**

- I don't have data
- I have a small dataset
- My data is not labelled (or only a small part)

The data problem

In 99% of cases, the main issue with using Deep Learning will be **data**

- I don't have data
- I have a small dataset
- My data is not labelled (or only a small part)
  - cannot work with Supervised Learning

The data problem

In 99% of cases, the main issue with using Deep Learning will be **data**

- I don't have data
- I have a small dataset
- My data is not labelled (or only a small part)
- My data is biased

The data problem

In 99% of cases, the main issue with using Deep Learning will be **data**

- I don’t have data
- I have a small dataset
- My data is not labelled (or only a small part)
- My data is biased
  - some classes rarely show up (class imbalance)
  - datapoint distribution is not uniform
  - missing data

Using Expert knowledge

If you/someone has knowledge about the data or task, use it!

Using Expert knowledge

If you/someone has knowledge about the data or task, use it!

Expert knowledge:

prior knowledge about the task that allows to restrict the search space

Using Expert knowledge

If you/someone has knowledge about the data or task, use it!

Expert knowledge:

prior knowledge about the task that allows to restrict the search space

- identify mistakes in data
- remove useless features / datapoints
- normalize and pre-process data
- structure in the data (inputs or outputs), ...

Using Expert knowledge

If you/someone has knowledge about the data or task, use it!

Expert knowledge:

prior knowledge about the task that allows to restrict the search space

- identify mistakes in data
- remove useless features / datapoints
- normalize and pre-process data
- structure in the data (inputs or outputs), ...

$\rightarrow$ *inductive biases* are an example of expert knowledge!

(I know that information in text/an image is local so i can use attention/convolution)

Data Augmentation to help with small datasets

Example of expert knowledge: **data augmentation**

(a) Original (b) Crop and resize (c) Crop, resize (and flip) (d) Color distort. (drop) (e) Color distort. (jitter)

(f) Rotate $\{90°, 180°, 270°\}$ (g) Cutout (h) Gaussian noise (i) Gaussian blur (j) Sobel filtering

Figure 4. Illustrations of the studied data augmentation operators. Each augmentation can transform data stochastically with some internal parameters (e.g. rotation degree, noise level). Note that we *only* test these operators in ablation, the *augmentation policy used to train our models* only includes *random crop (with flip and resize)*, *color distortion*, and *Gaussian blur*. (Original image cc-by: Von.grzanka)

$\rightarrow$ I **know** that all those still correspond to the same image

$\rightarrow$ LOTS more data!

Notion of Latent Space

Notion of Latent Space

What does the network actually learn?

Notion of Latent Space

What does the network actually learn?

$\rightarrow$ What do the *features* (neurons) at a given layer encode?

Notion of Latent Space

What does the network actually learn?

$\longrightarrow$ What do the *features* (neurons) at a given layer encode?

Intuition of learned latent space for language: (thousands of dimensions)

LLMs do this for entire sentences/prompts! $\longrightarrow$ space of *meaning*

Notion of Latent Space

Sometimes called

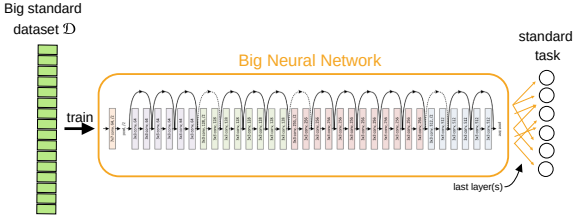
- *Latent* space
- *Embedding* space
- *Feature* space / learned features
- *Representation* space / learned representation

Fine-tuning with Transfer Learning

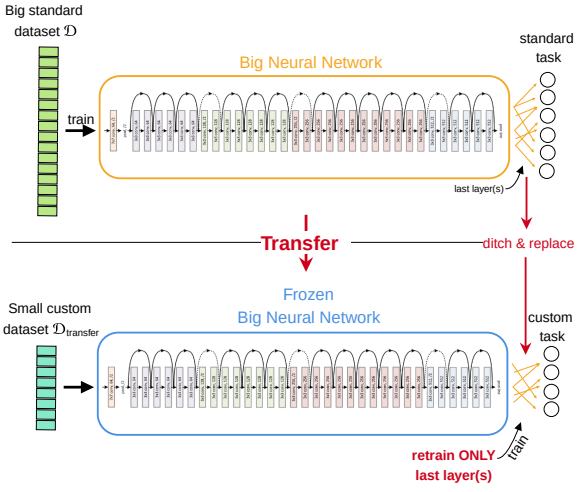
Little data?

Use *existing models* as a strong starting point and **transfer** knowledge (features)

Fine-tuning with Transfer Learning



Fine-tuning with Transfer Learning



The rise of Self-Supervision

Few/no labels, no prior info?

Learn useful features from *the input data alone*

The rise of Self-Supervision

Few/no labels, no prior info?

Learn useful features from *the input data alone*

Self-supervision:

train for generally useful *"pretext" tasks* / *pseudo-labels* (easy to generate)

Examples:

The rise of Self-Supervision

Self-supervision:

train for generally useful *"pretext" tasks* / *pseudo-labels* (easy to generate)

Examples:

- auto-encoder (input reconstruction)
- language:

The rise of Self-Supervision

Self-supervision:

train for generally useful *"pretext" tasks* / *pseudo-labels* (easy to generate)

Examples:

- auto-encoder (input reconstruction)
- language: similar word encoding, next token prediction, missing word prediction
- vision:

The rise of Self-Supervision

Self-supervision:

train for generally useful *"pretext" tasks* / *pseudo-labels* (easy to generate)

Examples:

- auto-encoder (input reconstruction)
- language: similar word encoding, next token prediction, missing word prediction
- vision: consistent prediction from augmented images

$\rightarrow$ hugely popular today!!

The rise of Self-Supervision

Self-supervision:

train for generally useful *"pretext" tasks* / *pseudo-labels* (easy to generate)

Examples:

- auto-encoder (input reconstruction)
- language: similar word encoding, next token prediction, missing word prediction
- vision: consistent prediction from augmented images

→ hugely popular today!!

→ perform **transfer** on the learned representation afterwards

Time for Presentations!

You know all you need to listen to the article presentations.